

Practical 6 - Demo Doc

Wonder Team

Miss. Keagan van Biljon - **u23594412**

Mr. Matthew Malan - **u25009801**

Mr. Lucian van der Merwe - **u25054262**

Miss. Hayley Nel - **u25101821**

Mr. Ruben van der Vyver - **u25007786**

[Github Repository](#)

Task 1:

A workflow is a series of repetitive, tedious tasks and manual processes. A workflow management system is a software that automates some or all of the tasks in a workflow, with the aim of improving productivity, and freeing up resources (personnel etc) to be used elsewhere. Work items are commonly represented as objects, as they can be used or worked on by multiple processes, and have their own properties. WorkItems are assigned and routed by use of the Strategy that allocates work items based on whatever strategy is needed. Approval, rejection and other states are handled or represented by moving processes into different areas of a hierarchy, or will simply be removed and added to a different hierarchy ([interfacing.com](#)). Most information on participants, roles and their responsibilities is hard to come by. On this topic, some workflow management softwares ([doforms.com](#)) model their users in a hierarchy. A user can have privileges to access certain workflows or even different parts of the software. Thus roles can be delegated through this system. Nifty ([niftypm.com](#)) Handles external integrations in a manner similar to how github works. They have a tool that allows you to add almost any external software into your “pipeline”, and then, similar to git, you can push or pull tasks and the workflow management software figures out where the information is supposed to be sent to or received from. Sensus ([sensus-processmanagement.com](#)) has a built in version manager that records multiple changes to a process or task, then it allows you to compare different versions and see what has been changed. It also maintains a central process library, which allows you to keep track of multiple processes and changes to these processes at once. Trello ([trello.com](#)) maintains a library of templates that can be reused across multiple different, but related, processes. Real workflow systems model nested activities as such: The outer layer of the activity must wait for the inner layer to finish. If the inner layer cannot complete then it throws a warning and waits for a user.

Task 2:

TIVIDY is a generic workflow management system used to define, execute and monitor workflows. It manages work items, participants, dependencies, routing, approvals, failures

and progress. Workflow definitions describe how a process should operate, while running instances track the progress and data of a specific process. TIVIDY should remain generic and general, without becoming too focused on a specific field/organisation.

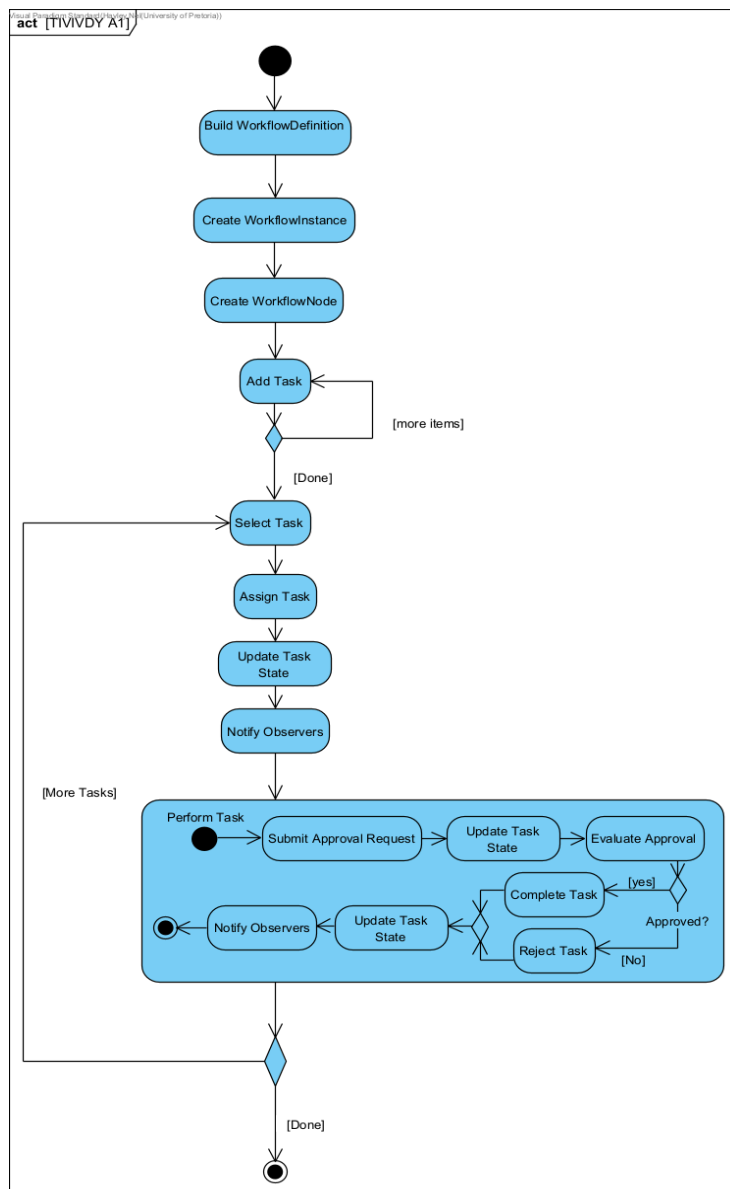
Our scenario:

A mid-size studio ships a content update. A feature request is approved by a producer and then splits into parallel work: art, code, audio and narrative. These join for integration and a build, then go to QA. Failed QA loops back to the responsible discipline. Then came creative-director sign-off, platform certification (Steam, PlayStation, Nintendo, each with its own API and rules), and release. It is approval-heavy, with lots of fork/join, rework loops, escalation when cert is rejected, and external systems (CI server, bug tracker, storefront).

Task 3:

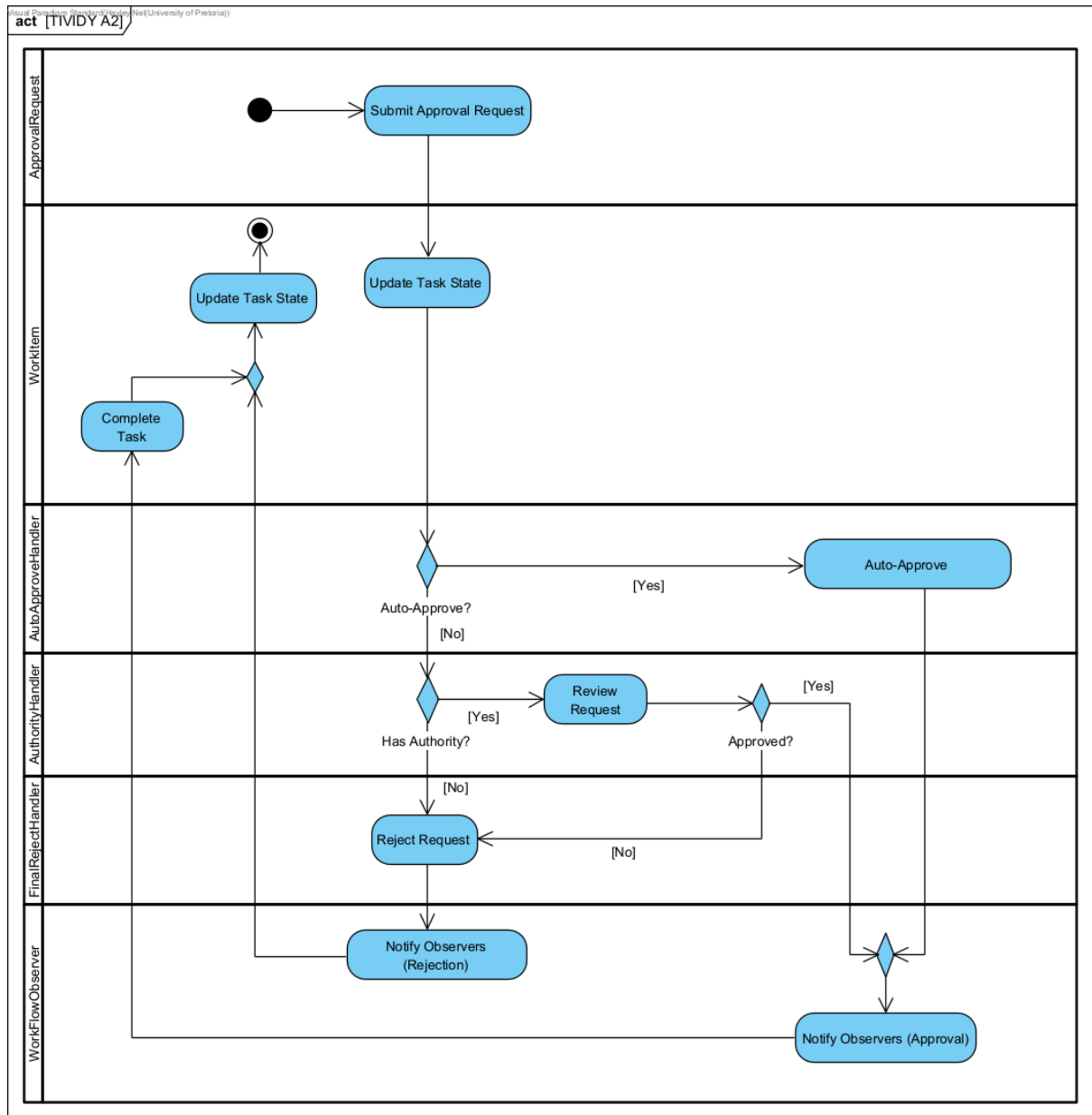
Activity Diagram 1:

Generic Workflow from creation to completion



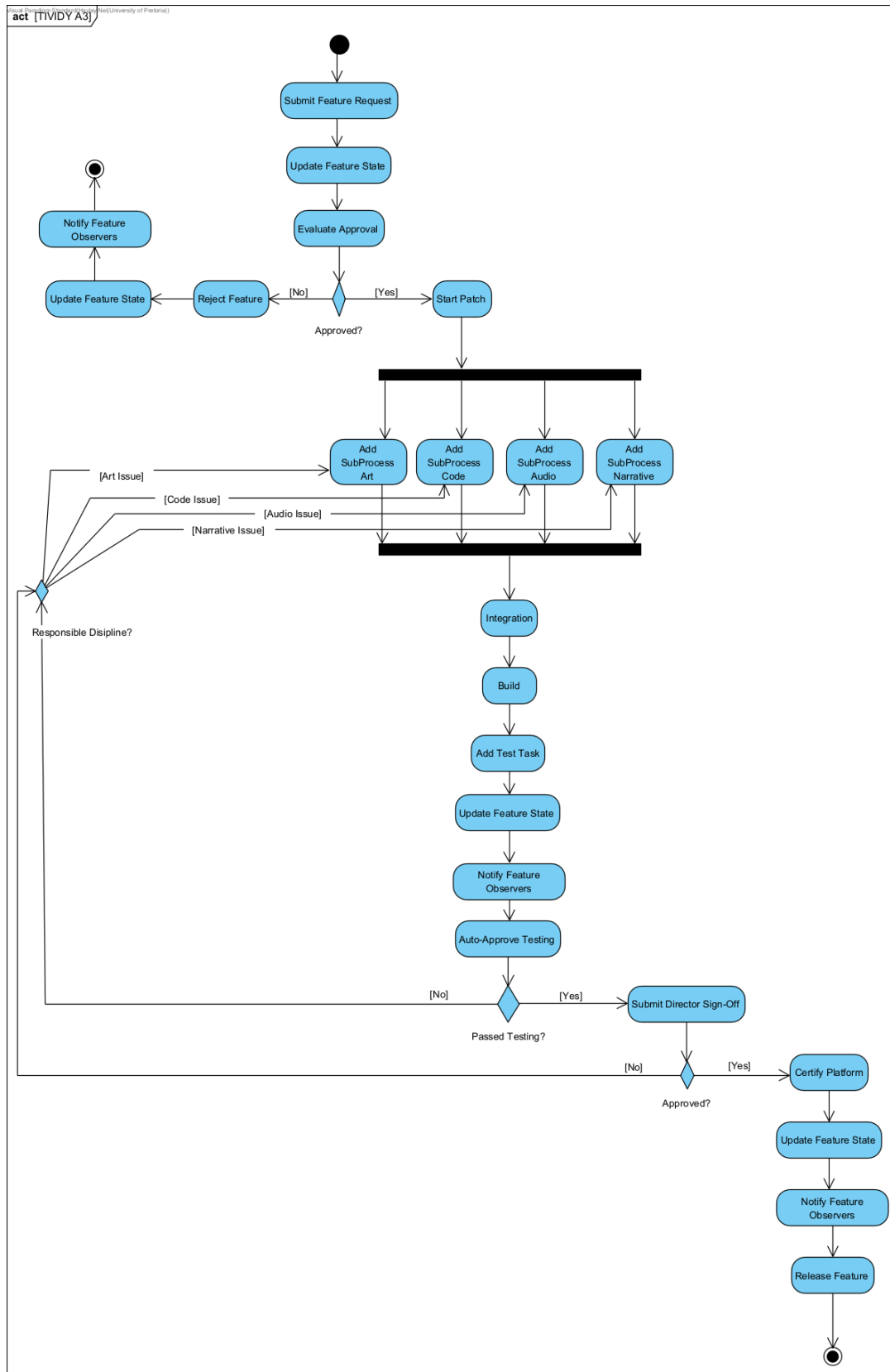
Activity Diagram 2:

TIVIDY Approval Workflow



Activity Diagram 3:

Scenario A Workflow



Task 4:

Composite -

Problem: Workflows contain stages, tasks and sub-processes, nested arbitrarily

Using the Composite design pattern, we give the workflow system a structured hierarchy. WorkflowNode, CompositeNode and Task as Component, Composite and leaf. Two additional “concrete” composite classes are made: Stage and SubProcess to further define actions/ rules specific to the situation.

Decorator -

Problem: Tasks gain validation, priority, audit or security without new subclasses

We use the Decorator design pattern to add more functionality and information to tasks without editing the task itself. We achieve this with the classes Task, TaskDecorator and concrete decorators. Since, in our design, Task acts as its own component class which has different children.

State -

Problem: Work item lifecycle: valid operations depend on state

Using the State design pattern we allow the tasks to have defined states which can be reacted to. This allows additional states to be added, existing states to be edited and maintained. By removing this from the Task class itself, it allows more versatility when expanding upon the design.

Chain of Responsibility -

Problem: Multi-level approval and escalation

Using the Chain of Responsibility design pattern allows a line of approval to be created, maintained and edited. By doing so, any additions can be added easily and each stage can set new rules and requirements depending on the task.

Observer -

Problem: Many parties react to the same workflow event

By using the Observer design pattern, we create a system which allows changes/events in workflow instances to be reacted to. A workflow instance inherits from (is a) subject in the Observer design pattern. Any updates made will be sent directly to any number of observers attached such as AuditLog, NotificationService, etc.

Command -

Problem: Actions on work items, recorded for audit and undo

By using the Command design pattern, we turn each action on a work item into its own object that can be executed and reversed. A history keeps every executed command, giving a full audit trail and allowing undo. This prevents changes from being applied untracked or without a way to revert them.

Builder -

Problem: Constructing a valid definition tree step by step

By using the Builder design pattern, we define the set definition of a valid tree and the steps to create one. This prevents broken or invalid trees from being created.

Strategy -

Problem: Assignment and routing rules differ per organisation

By using the Strategy design pattern we describe and define processes for different assignment and routing rules, allowing a specific rule to change during runtime, where necessary. By using this design pattern, we allow additional strategies to be created and the current strategies to be edited and maintained without interfering with existing code. This allows less generic implementation for future uses.

Facade -

Problem: Simple entry point for scenarios/main

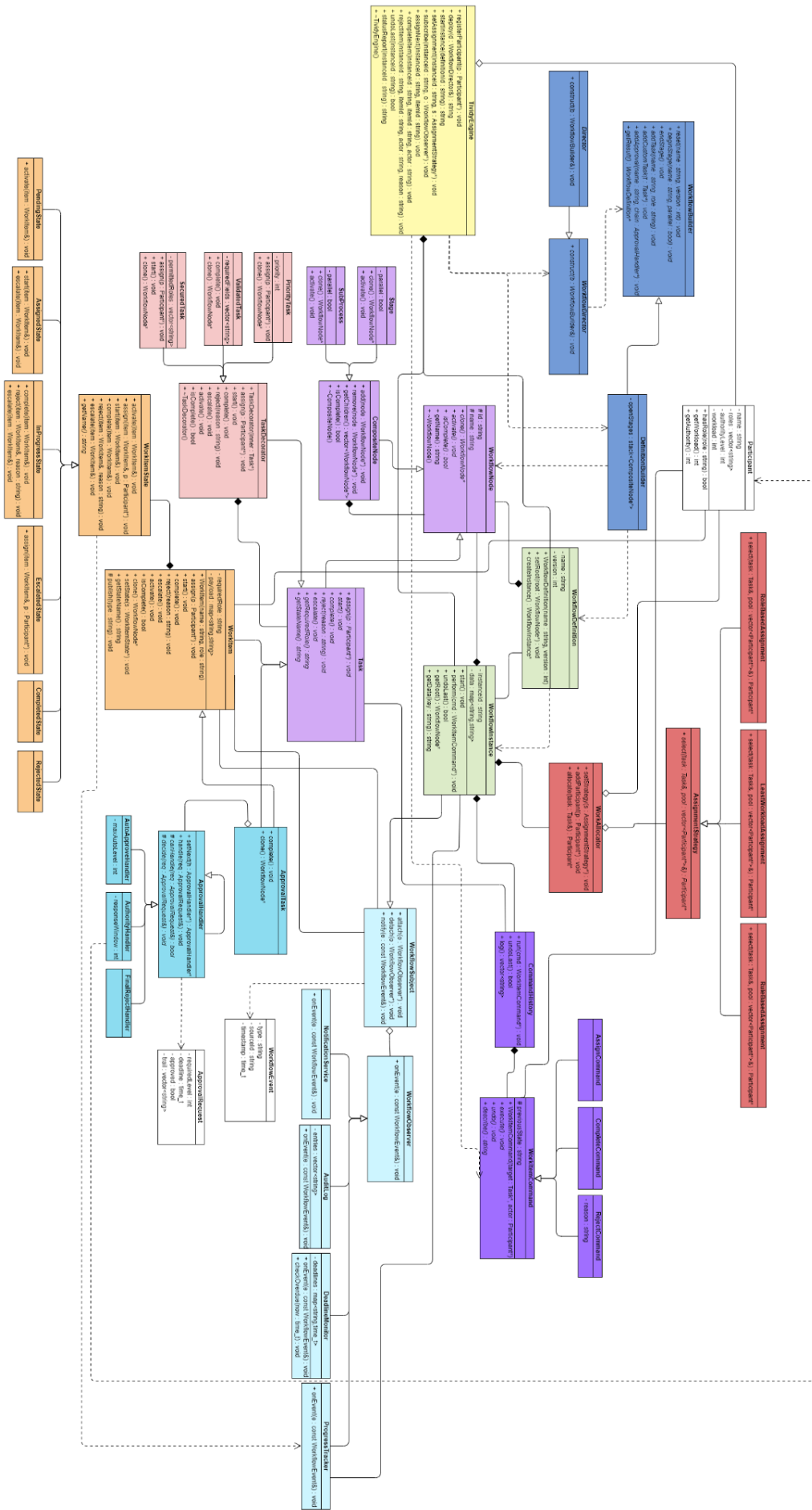
By using the Facade design pattern, we abstract the entire design further for the user's accessibility. This allows a complicated system to have a simple entry point with defines operations, without having to understand how every class connects with one another.

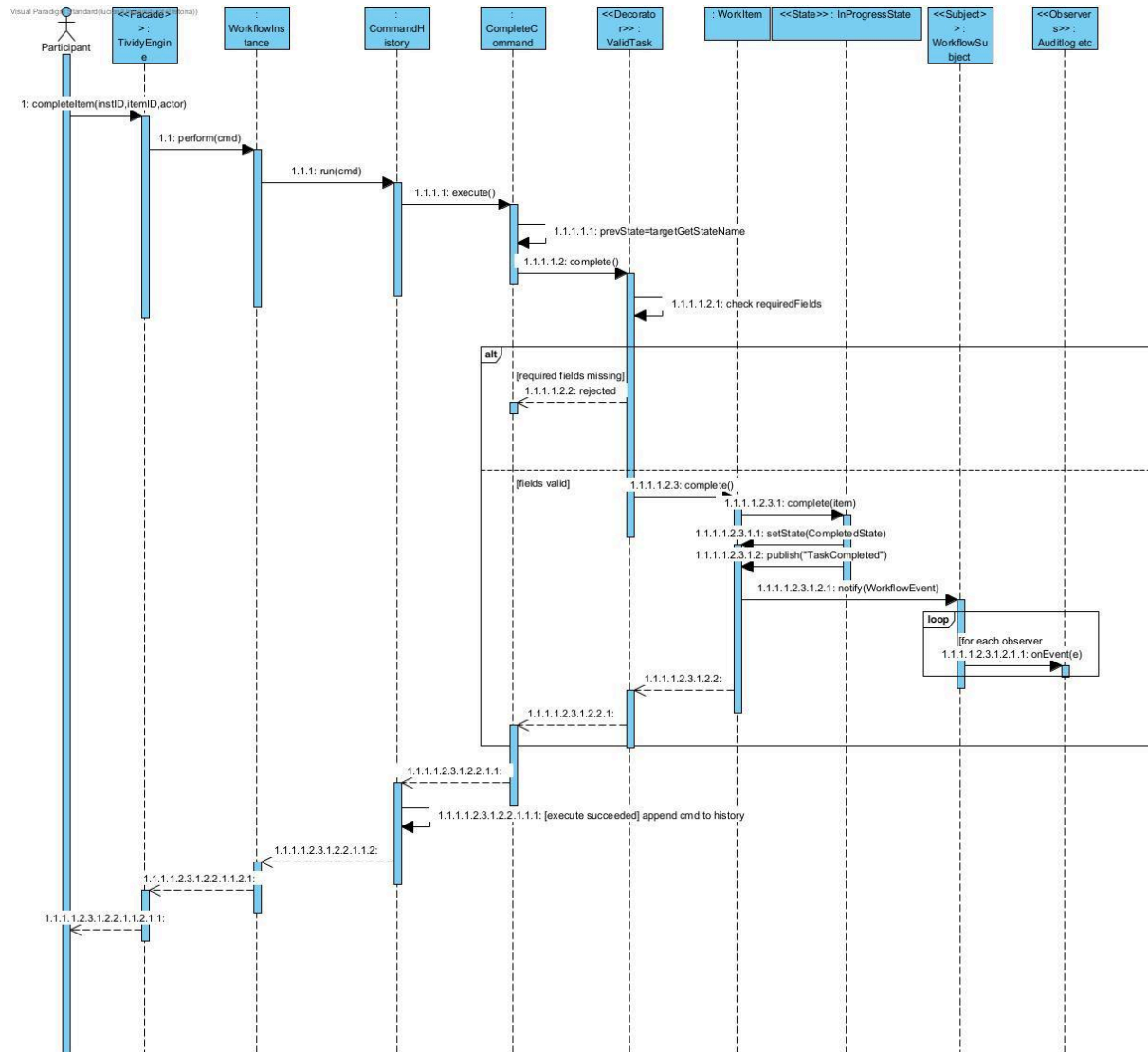
Prototype -

Problem: Many independent instances from one definition

By using the Prototype design pattern to easily produce copies of workflows without complicated creations from scratch. This prevents unnecessary overhead on the user end, allowing creation without fully understanding the underlying structure.

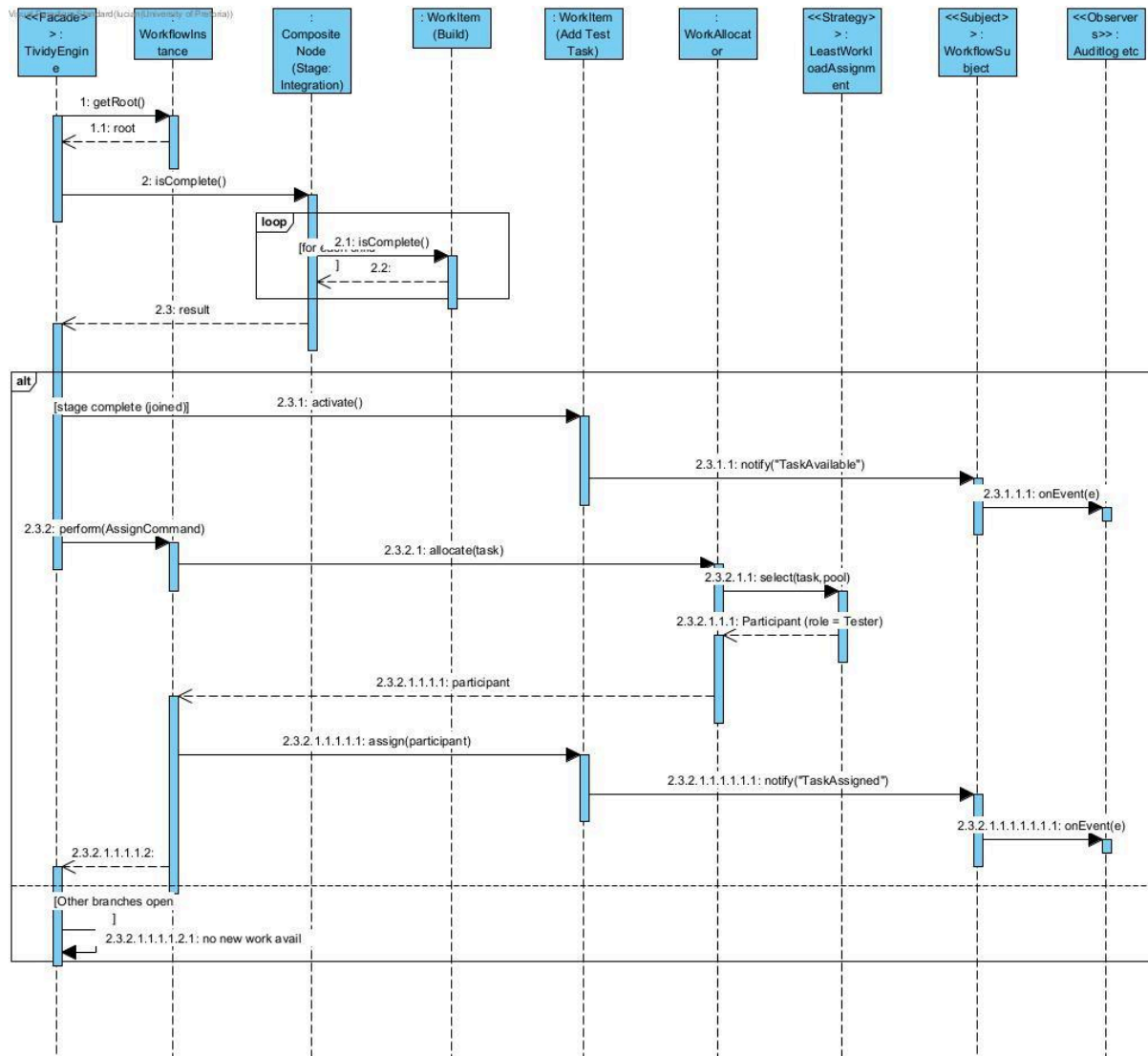
Task 5:





Sequence Diagram 2:

Determining what happens next.



State Diagram 1:

WorkItemState.

Visual Paradigm Standard(lucian(University of Pretoria))

